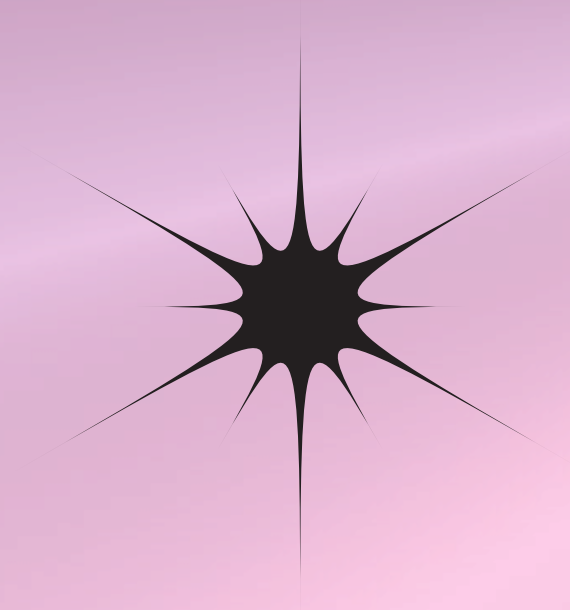


Universidade Federal De Roraima
DCC403 - Sistemas Operacionais
Projeto Final

FORMALRACE-CHECKER

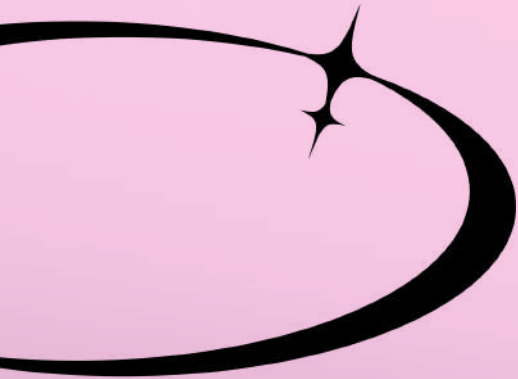


UM VERIFICADOR FORMAL DE CORRIDAS DE DADOS
PARA CÓDIGO CONCORRENTE EM C



Discente: Sarah Caroline Amaral Pereira.
Boa Vista - RR
2026.1

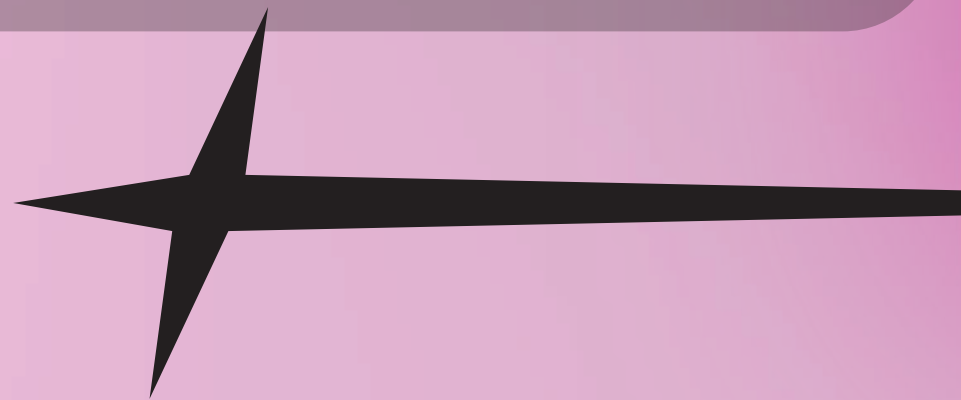
O que é o FormalRace-Checker?



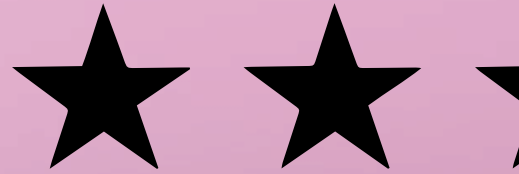
É uma ferramenta de **análise estática**, que lê e analisa códigos em C com **múltiplas threads (POSIX Threads/ pthreads)**, e detecta corridas de dados usando matemática formal, **SAT Solving**.

Ele resolve o problema de que programas concorrentes podem ter bugs invisíveis, como as **corridas de dados(data races)**, definidas por acessos simultâneos à mesma memória **sem sincronização**, que geram resultados imprevisíveis e são impossíveis de detectar com testes convencionais.

O **FormalRace-Checker** prova matematicamente se o bug existe ou não, sem precisar executar o programa.



Tecnologias utilizadas no projeto



- **C++11** como linguagem de implementação, por ter suporte nativo a `std::regex`, `std::vector`, `std::string`;
- **std::regex**, uma biblioteca nativa do C++, que reconhece padrões no código C sem dependências externas;
- **MiniSAT**, um SAT Solver open source, que resolve fórmulas CNF e responde SAT ou UNSAT;
- **Make**, automação de build, que compila os 4 módulos com um único comando;
- **pthread**, uma biblioteca POSIX, que é o padrão analisado pela ferramenta nos arquivos de entrada.

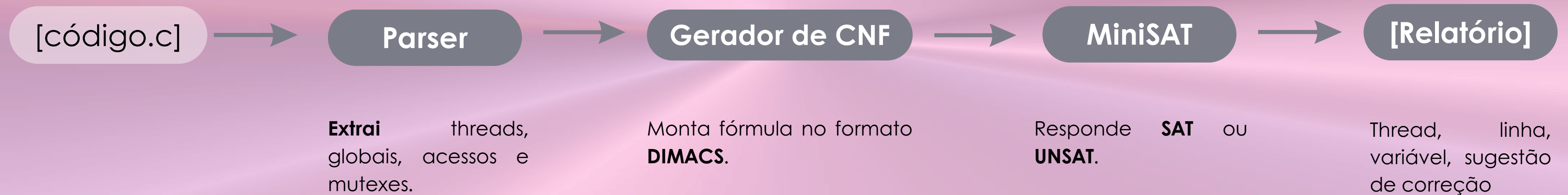
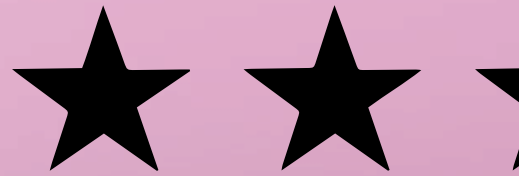


Transformando o problema em fórmula booleana

$$\text{Race}(W, R) \Leftrightarrow W \wedge R \wedge \neg P(W, R)$$

- **W** = Thread 1 realiza escrita na variável global;
 - **R** = Thread 2 realiza leitura na mesma variável;
 - **P** = existe mutex protegendo ambos os acessos.
-
- **SAT** se existe uma execução com race: **RACE DETECTADO**;
 - **UNSAT** se não existe tal execução: **CÓDIGO SEGURO**.

Pipeline do FormalRace-Checker



Módulo 1 (Parser)- Como a ferramenta lê o código

Os padrões são identificados por expressões regulares via **std::regex**.

```
if (bracketBalance == 0) {  
    smatch match;  
    if (regex_search(l, match, globalVarRegex)) {  
        string tipo = match[1];  
        string nome = match[2];  
        // Filtra nomes genéricos de parâmetro e tipo void  
        if (nome != "arg" && nome != "args" && tipo != "void") {  
            variaveisGlobais.insert(nome);  
        }  
    }  
}
```

```
regex globalVarRegex(  
    R"((int|float|double|char|long|short|unsigned|signed)\s+)"  
    R"(([a-zA-Z_][a-zA-Z0-9_]*)\s*[=;])"  
);
```

Faz 3 passagens:

1. Encontra variáveis globais, declaradas fora de funções;
2. Encontra funções de thread, terceiro argumento do pthread_create;
3. Dentro de cada thread: registra leitura/escrita, linha e mutex ativo.

Módulo 2 (Arquivo CNF)- Traduzindo código em matemática

O que é o formato DIMACS:

É o formato padrão de entrada do MiniSAT. Cada variável é um número inteiro. Cada linha é uma cláusula, uma condição que precisa ser satisfeita. O arquivo tem sempre um cabeçalho e depois as cláusulas.

Cabeçalho: **p cnf** <num_variáveis> <num_clausulas>

Para cada **par (escrita W, leitura R)** de threads diferentes na **mesma variável**:

Se **mutex de W $\neq$ mutex de R**, ou um deles está vazio:
→ par inseguro → gera cláusulas

Se o **MiniSAT** encontra valores que satisfazem todas as cláusulas ao mesmo tempo retorna: SAT → race confirmado.

p cnf 3 3 ← 3 variáveis, 3 cláusulas

1 0 ← variável 1 = escrita (W)
deve ser verdadeira

2 0 ← variável 2 = leitura (R)
deve ser verdadeira

-3 0 ← variável 3 = proteção (P)
deve ser FALSA (sem mutex)

Módulos 3 e 4- Do solver ao relatório



Módulo 3: Solver Connector

- Lê o cabeçalho do CNF e verifica se há cláusulas;
- Se zero cláusulas, retorna UNSAT direto, sem chamar o MiniSAT;
- Se há cláusulas, invoca o MiniSAT via system();
- Lê o arquivo de resultado e retorna o veredito.

Módulo 4: Relatório

Quando **SAT** (race detectado), exibe:

- Todos os acessos extraídos (thread, variável, linha, mutex);
- Par conflitante: qual thread escreve, qual lê, em qual linha;
- Confirmação da ausência de mutex comum;
- Sugestão de correção com exemplo de código.

Quando **UNSAT** (código seguro), exibe:

- Confirmação formal de que nenhum race foi encontrado;
- Código aprovado para execução concorrente.



Cenários de testes obrigatórios

- **Cenário A(Sem proteção):** Escrita sem mutex, leitura sem mutex; Esperado: SAT → data race confirmado;
- **Cenário B(Proteção correta):** Mesmo mutex em todos os acessos; Esperado: UNSAT → código seguro;
- **Cenário C(Múltiplos locks):** Variáveis diferentes com mutexes diferentes; Esperado: UNSAT → código seguro.

Demonstração no terminal Linux:

```
./formalrace-checker tests/cenario_a.c  
./formalrace-checker tests/cenario_b.c  
./formalrace-checker tests/cenario_c.c
```

Limitações

- Não detecta acesso via ponteiros (*p = 42);
- Não analisa locks dentro de funções auxiliares chamadas pela thread;
- Escopo adequado para código de pequeno e médio porte com sincronização explícita.

- **Repositório:**

github.com/sarahw3b/FP_DCC403_Tema_8_UFRR

Obrigada.

